

Tomaten-Bewässerungswächter auf dem Raspberry Pi 3B+

Vollständige Dokumentation: Aufbau, Installation, Betrieb, Fehlersuche.

Stand 18.08.2026. Gerät in Betrieb genommen, im Kaltstart geprüft, Display angeschlossen und laufend.

1. Was das Gerät tut

Eine ESP32-CAM beobachtet eine Tomatenpflanze. Der Raspberry Pi holt alle fünf Minuten ein Bild über Modbus TCP ab, trennt die Pflanze über ihren Grünanteil vom Hintergrund und vergleicht zwei Kennwerte mit einer Referenz, die nach dem Gießen gesetzt wird: die sichtbare Grünfläche und die Höhe des Schwerpunkts der Grünmaske. Welke Blätter hängen durch — die Fläche schrumpft, der Schwerpunkt sinkt. Daraus entsteht eine Ampel.

```
ESP32-CAM  —WLAN—>  Raspberry Pi 3B+  —>  Webseite (Port 8083)
(Solar, Akku)                pflanzen_dashboard.py  —>  OLED 128x64 (I2C)
                               als Systemdienst       —>  pflanzen_log.csv
```

Der Pi ersetzt den PC, der diese Aufgabe vorher übernommen hatte. Er läuft unbeaufsichtigt durch, startet nach Stromausfall von selbst und braucht weder Monitor noch Tastatur.

2. Gerätedaten und Zugänge

Rechner	Raspberry Pi 3 Model B+, 1 GB
Betriebssystem	Raspberry Pi OS (32-bit) mit Desktop, Debian-Basis Trixie
Netzwerkname	tomate
Adresse	<IP-des-Pi> (per DHCP, kann sich ändern)
Benutzer	pi
Kennwort	xxxxxxxxxx
WLAN	<WLAN-Name>
Programmordner	/home/pi/tomate
Dienst	pflanzen_waechter
Dashboard	http://<IP-des-Pi>:8083
Startverhalten	Textanzeige (kein lokaler Desktop)
Fernzugriff	SSH, xrdp (Windows-Remotedesktop), VNC eingeschaltet

Zum Kennwort: xxxxxxxxx ist die historische Werkseinstellung und damit das Erste, was automatische Suchprogramme durchprobieren. Solange kein Zugang von außen auf den Pi weitergeleitet wird, bleibt das Risiko im Heimnetz. Ändern lässt es sich jederzeit mit passwd.

3. Warum die Einrichtung so und nicht anders

Drei Entscheidungen prägen den Aufbau. Sie sind hier begründet, damit sie bei einer Neuinstallation nicht neu erarbeitet werden müssen.

32 Bit statt 64 Bit

Der Prozessor des 3B+ ist 64-bit-fähig, das Betriebssystem liefere. 64-Bit-Code belegt aber mehr Arbeitsspeicher, weil jede Adresse doppelt so breit ist. Bei 1 GB und laufendem Desktop führt das zu ständigem Auslagern auf die Speicherkarte: das Gerät wird zäh, die Karte verschleißt. Die Empfehlung der Raspberry Pi Foundation lautet für Geräte mit 1 GB entsprechend 32 Bit; 64 Bit lohnt ab dem Pi 4 mit mehr Speicher. Für Python, numpy und Pillow ist die Wahl ohne Belang.

Desktop-Ausgabe, aber Start in die Textanzeige

Die Desktop-Ausgabe wurde gewählt, weil ein Monitor zur Verfügung stand und die Ersteinrichtung damit sichtbar wird — jeder Fehler steht im Klartext auf dem Schirm, statt von außen erraten werden zu müssen.

Der Pi startet trotzdem nicht in den Desktop, sondern in die Textanzeige (raspi-config, Boot-Verhalten B2). Das hat zwei Gründe. Erstens bleiben rund 300 MB Arbeitsspeicher frei, solange niemand zuschaut. Zweitens — und das ist der zwingende Grund — kollidiert eine automatisch angemeldete Desktop-Sitzung mit xrdp: Der Fern-Desktop bricht dann unmittelbar nach dem Verbindungsaufbau wieder ab. Ohne lokale Sitzung läuft xrdp zuverlässig und erzeugt den Desktop genau dann, wenn man sich verbindet.

Eigener Displaytreiber statt Fremdbibliothek

oled_anzeige.py spricht das OLED unmittelbar über /dev/i2c-1 an, mit einem ioctl-Aufruf und os.write. Der Grund: Raspberry Pi OS lässt seit Bookworm keine pip install-Installationen mehr ins System (externally managed environment). Bibliotheken wie luma.oled oder Adafruit-Blinka müssten in einer virtuellen Umgebung installiert werden, was den Dienststart verkompliziert. Das Display braucht ohnehin nur Schreibzugriffe — die beherrscht der Kernel-Treiber i2c-dev direkt. Gezeichnet wird mit Pillow, das für die Bildauswertung sowieso vorhanden ist. Damit hat die Anzeige keine einzige Zusatzabhängigkeit.

4. Installation von Null

Diese Reihenfolge hat sich bewährt. Sie unterscheidet sich von der üblichen Anleitung, weil zwei Fehler im Raspberry Pi Imager umgangen werden (siehe Abschnitt 8).

4.1 Imager

Nicht Version 2.0.10 verwenden. Diese Fassung hat zwei belegte Fehler: sie stellt Schaltflächenbeschriftungen verstümmelt dar (jeder Buchstabe um zwei Stellen verschoben, aus SPEICHERN wird QNCGAF CPL), und sie schreibt die Voreinstellungen teilweise oder gar nicht auf die Karte, ohne das zu melden.

Verwendet wurde Version 1.9, zu beziehen über die Veröffentlichungsseite github.com/raspberrypi/rpi-imager/releases.

4.2 Karte brennen — ohne Voreinstellungen

- Gerät: Raspberry Pi 3
- Betriebssystem: Raspberry Pi OS (32-bit), die Fassung mit Desktop (nicht Lite, nicht Full, nicht Legacy)
- Speichermedium: die SD-Karte (64 GB, Klasse 10)
- Schreiben, und bei der Frage nach den Voreinstellungen ausdrücklich NEIN

Die Voreinstellungen werden nicht gebraucht, weil die Einrichtung am Monitor erfolgt. Damit ist die fehleranfälligste Stelle des Imagers umgangen.

Während des Schreibens fragt Windows möglicherweise, ob ein Datenträger formatiert werden soll. Immer Abbrechen. Die Frage bezieht sich auf die Linux-Partition, die Windows nicht lesen kann, und kommt bei jedem Einstecken.

4.3 Erster Start am Monitor

Monitor per HDMI, Tastatur und Maus anschließen, dann erst Karte einlegen und Strom anlegen. Netzteil mit mindestens 2,5 A verwenden.

Der Begrüßungsassistent führt durch:

Schritt	Eingabe
Land, Sprache, Zeitzone	Germany, German, Berlin — Häkchen „Use English language“ weglassen
Benutzer	Name pi, Kennwort xxxxxxxxx
WLAN	<WLAN-Name> und Kennwort
Browser	Chromium
Software aktualisieren	kann übersprungen werden; bricht der Schritt ab, ist das folgenlos

Anschließend neu starten lassen.

4.4 Schnittstellen einschalten

Himbeer-Menü → Einstellungen → Raspberry-Pi-Konfiguration → Reiter Schnittstellen. Dort einschalten:

- SSH — Fernzugriff über die Kommandozeile
- VNC — Fern-Desktop (teilt sich die lokale Sitzung)
- I2C — Leitung zum OLED-Display

Danach neu starten.

4.5 Netzwerknamen setzen

Über SSH oder am Monitor:

```
sudo raspi-config nonint do_hostname tomate
sudo reboot
```

Damit heißt das Gerät tomate und ist als `tomate.fritz.box` ansprechbar, sofern die Fritz!Box den Namen weiterreicht. Die Zahlenadresse funktioniert immer.

4.6 Dateien übertragen

Auf dem Pi den Ordner anlegen:

```
mkdir -p ~/tomate
```

Auf dem Windows-PC eine zweite PowerShell öffnen — nicht die, in der die SSH-Sitzung läuft, denn dort würden die Befehle auf dem Pi ausgeführt. In den Ordner mit den Dateien wechseln, etwa:

```
cd $HOME\Downloads
```

Dann die Dateien einzeln übertragen. Einzeln deshalb, weil lange Zeilen beim Einfügen in die PowerShell umbrechen und dann als zwei unvollständige Befehle ausgeführt werden:

```
scp pflanzen_dashboard.py pi@<IP-des-Pi>:/home/pi/tomate/  
scp oled_anzeige.py pi@<IP-des-Pi>:/home/pi/tomate/  
scp waveshare_io.py pi@<IP-des-Pi>:/home/pi/tomate/  
scp waveshare_pi_test.py pi@<IP-des-Pi>:/home/pi/tomate/  
scp waveshare_wandler_test.py pi@<IP-des-Pi>:/home/pi/tomate/  
scp pflanzen_waechter.service pi@<IP-des-Pi>:/home/pi/tomate/  
scp install_pi.sh pi@<IP-des-Pi>:/home/pi/tomate/  
scp ANLEITUNG_Pi_OLED.md pi@<IP-des-Pi>:/home/pi/tomate/
```

Jede Zeile fragt nach dem Kennwort. Wer lieber mit der Maus arbeitet, nimmt WinSCP (Protokoll SFTP, Rechner <IP-des-Pi>, Benutzer pi).

4.7 Einrichten

Im SSH-Fenster:

```
cd ~/tomate  
chmod +x install_pi.sh  
sudo ./install_pi.sh
```

Das Skript führt fünf Schritte aus:

1. installiert python3-numpy, python3-pil, fonts-dejavu-core, i2c-tools über apt (nicht über pip),
2. schaltet I2C ein und trägt 400 kHz Bustakt in /boot/firmware/config.txt,
3. sucht das Display auf dem Bus und meldet das Ergebnis,
4. richtet den Systemdienst pflanzen_waechter ein und schaltet ihn scharf,
5. startet ihn und zeigt seinen Zustand.

Solange kein Display angeschlossen ist, meldet Schritt 3, dass keines gefunden wurde. Das ist kein Fehler — die Anzeige ist Zugabe, das Programm läuft ohne sie unverändert.

4.8 Start in die Textanzeige umstellen

```
sudo raspi-config nonint do_boot_behaviour B2  
sudo reboot
```

Danach zeigt der Monitor nur noch eine Eingabeaufforderung. Der Wächter läuft davon unberührt, denn er ist ein Systemdienst und startet vor jeder Anmeldung.

4.9 Fern-Desktop

```
sudo apt install -y xrdp
```

Am Windows-PC mstsc starten, <IP-des-Pi> eingeben, im Anmeldefenster von xrdp Session Xorg, Benutzer pi, Kennwort xxxxxxxxxx.

Wichtig: Ohne die Umstellung aus 4.8 bricht die Verbindung sofort wieder ab.

5. Betrieb

Das Dashboard ist im Browser jedes Geräts im Heimnetz erreichbar:

<http://<IP-des-Pi>:8083>

Ablauf im Alltag: Pflanze gießen, im Dashboard einmal Referenz setzen. Von da an meldet der Wächter, wenn die Blätter hängen. Die Referenz sollte bei ähnlicher Beleuchtung gesetzt werden wie später gemessen wird.

Befehle

Zweck	Befehl
Läuft der Wächter?	<code>systemctl status pflanzen_waechter</code>
Mitlesen, was er tut	<code>journalctl -u pflanzen_waechter -f</code> (Ende mit Strg+C)
Anhalten	<code>sudo systemctl stop pflanzen_waechter</code>
Starten	<code>sudo systemctl start pflanzen_waechter</code>
Nach Änderung neu laden	<code>sudo systemctl restart pflanzen_waechter</code>
Autostart abschalten	<code>sudo systemctl disable pflanzen_waechter</code>
Adresse des Pi	<code>hostname -I</code>
Sauber ausschalten	<code>sudo poweroff</code>

Den Pi nicht einfach vom Strom trennen — dabei kann die Speicherkarte Schaden nehmen. Nach `sudo poweroff` blinkt die grüne Leuchtdiode zehnmal und bleibt dann dunkel; erst dann das Netzteil ziehen.

Einstellungen

Alle Stellschrauben stehen oben in `pflanzen_dashboard.py`. Ändern mit `nano ~/tomate/pflanzen_dashboard.py` danach `sudo systemctl restart pflanzen_waechter`.

Größe	Bedeutung
<code>ESP32CAM_IP</code>	Name oder Adresse der Kamera
<code>INTERVALL</code>	Sekunden zwischen zwei Bildern, Vorgabe 300
<code>EXG_SCHWELLE, MIN_GRUEN</code>	Empfindlichkeit der Grünerkennung
<code>GELB_FLAECHEN, ROT_FLAECHEN</code>	Flächenabfall in Prozent für Gelb und Rot
<code>GELB_ABSINKEN, ROT_ABSINKEN</code>	Absinken des Schwerpunkts für Gelb und Rot
<code>GLAETTUNG</code>	Median über die letzten n Messungen
<code>OLED_AKTIV</code>	False schaltet das Display ab
<code>OLED_TREIBER</code>	<code>ssd1306</code> oder <code>sh1106</code>
<code>OLED_KOPFSTEHEND</code>	True dreht die Anzeige um 180 Grad
<code>OLED_SEITENZEIT</code>	Sekunden je Anzeigeseite
<code>BILDER_TAGE</code>	Bilder älter als n Tage löschen, 0 = nie

Zu `BILDER_TAGE`: bei einem Bild alle fünf Minuten fallen rund 300 Bilder oder 4 MB pro Tag an. Ohne Aufräumen läuft die Karte in einem Jahr voll; die Vorgabe von 14 Tagen hält den Ordner bei etwa 60 MB.

6. OLED-Display

Angeschlossen und in Betrieb seit dem 18.08.2026. Verwendet wird ein weißes 128×64-Modul mit SSD1306-Controller, I2C, in der vierpoligen 0,96"-Bauform ohne Reset-Anschluss. Es war dafür nichts zu konfigurieren — I2C war bereits eingeschaltet, und das Programm sucht das Display beim Start auf beiden möglichen Adressen selbst.

Vor dem Anschließen den Pi sauber herunterfahren (`sudo poweroff`, grüne Leuchtdiode blinkt zehnmal, dann erst das Netzteil ziehen). Umstecken unter Spannung ist die häufigste Ursache für zerstörte Module.

Das Programm sucht das Display nur beim Start. Wird es im laufenden Betrieb angesteckt, bleibt es dunkel bis `sudo systemctl restart pflanzen_waechter`.

Display	Pi-Stiftleiste	Bezeichnung im Pinplan
VCC	Pin 1	3V3 Power
SDA	Pin 3	GPIO 2 (SDA)
SCL	Pin 5	GPIO 3 (SCL)
GND	Pin 9	Ground

So ausgeführt. Pin 1 ist die Ecke mit der eckigen Lötinsel, dem SD-Kartenschacht am nächsten. Die Masse liegt hier bewusst auf Pin 9 und nicht auf dem ebenfalls möglichen Pin 6: dann liegen alle vier Anschlüsse in derselben Reihe. Dabei die Zählweise beachten — in dieser Reihe folgen die ungeraden Nummern 1, 3, 5, 7, 9 aufeinander, Masse ist also der fünfte Stift, nicht der vierte. Pin 7 ist GPIO 4 und bleibt frei.

Die Stiftreihenfolge auf dem Displaymodul ist nicht einheitlich — verbreitet sind GND VCC SCL SDA und VCC GND SCL SDA. Die Beschriftung auf der Platine lesen, nicht nach Gewohnheit stecken. Vertauschte Versorgung überlebt das Modul meist nicht; vertauschte Datenleitungen sind harmlos, dann wird das Display lediglich nicht gefunden.

Nach dem Einschalten prüfen:

```
i2cdetect -y 1
```

In der Tabelle muss 3c erscheinen, bei manchen Modulen 3d. Beide findet das Programm von selbst. Anschließend:

```
sudo systemctl restart pflanzen_waechter
```

Was das Display zeigt

Drei Seiten wechseln sich alle sechs Sekunden ab. Der Inhalt verschiebt sich dabei um einen Bildpunkt, das beugt dem Einbrennen vor.

Seite 1, Pflanze:

```
Tomate          19:40
-----
[ GIESSEN! ]
Gruen 12.3 %
Abfall 9.4% Sinkt 2.1%
Ref 14.08. 19:30
```

Die große Zeile ist die eigentliche Aussage: Wasser ok, bald giessen oder GIESSEN!. Der Gießbefehl wird hell hinterlegt dargestellt und ist dadurch aus einigen Metern erkennbar. Weitere Texte: Nacht (zu dunkel zum Auswerten), keine Referenz und keine Pflanze.

Seite 2, Technik: Bildnummer und -größe, Betriebszeit der Kamera, WLAN-Pegel, freier Speicher der Kamera, Wiederholungen und verworfene Bilder, unten die eigene Adresse.

Seite 3, Verlauf: Flächenabfall als durchgezogene Linie, Absinken gepunktet.

Ein (!) in der Kopfzeile bedeutet: gerade keine Verbindung zur Kamera. Im Solarbetrieb ist das normal und verschwindet von selbst.

Das Display einzeln prüfen, ohne den Wächter:

```
sudo systemctl stop pflanzen_waechter
python3 ~/tomate/oled_anzeige.py demo
```

Zeigt alle drei Seiten mit erfundenen Werten. Ende mit Strg+C, danach `sudo systemctl start pflanzen_waechter`.

6b. Klemmenebene: Waveshare Modbus RTU Module (A)

Angeschlossen über einen Waveshare „USB TO 4CH RS485“ (Baustein WCH CH344) am USB des Pi. Das Modul hängt an Kanal 1, erkennbar an der Endung -if00 unter `/dev/serial/by-id/`. Verdrahtung A auf A, B auf B, Masse mitverbinden; Moduladresse 1, 9600 Baud, 8N1. Das Modul braucht eine eigene Spannungsversorgung.

Benötigt wird `python3-serial` (installiert das Einrichtungsskript nicht mit, daher bei einem Neuaufbau nachholen):

```
sudo apt install -y python3-serial
```

Auf einem RS485-Bus darf nur ein Master sprechen. Solange der Pi Master ist, müssen PC-Programme und das RS485-TO-ETH-Gateway () schweigen.

Im Dashboard erscheint eine eigene Karte: Analogeingänge im Sekundentakt mit Messbereich, Digitaleingänge als Anzeige, Relais 1 als Schaltknopf, Relais 2 als Gießaktor mit Zeitvorgabe, und die beiden Analogausgänge mit Eingabefeld in Mikroampere.

Der Messbereich der Analogeingänge wird beim Verbinden aus `waveshare_io.py` ins Modul geschrieben (AI_MODUS, Vorgabe 0 = 0-10 V). Für Spannungsmessung müssen zusätzlich die Schiebeschalter der beiden AI-Kanäle auf dem Modul auf OFF stehen. Änderungen an dieser Konstante wirken erst nach `sudo systemctl restart pflanzen_waechter`.

Einzeln prüfen lässt sich das Modul nur bei angehaltenem Dienst, weil dieser die serielle Schnittstelle dauerhaft belegt:

```
sudo systemctl stop pflanzen_waechter
python3 ~/tomate/waveshare_pi_test.py
sudo systemctl start pflanzen_waechter
```

7. Fehlersuche

Kamera

Immer „keine Verbindung zur Kamera“: Die ESP32-CAM nimmt nur eine einzige Modbus-Verbindung an. Solange das alte Wächterprogramm auf dem PC läuft, bleibt der Pi ausgesperrt — den PC-Wächter beenden. Im Solarbetrieb schaltet die Versorgung bei 10 % Akku ab und bei 50 % wieder ein; Aussetzer sind daher normal. Prüfen mit `ping tomatencam.fritz.box`.

Netzwerk

Der Pi ist unter seinem Namen nicht erreichbar: `.local` löst Windows ohne Zusatzdienste nicht auf. `tomate.fritz.box` versuchen, sonst die Zahlenadresse.

Adresse unbekannt: Raspberry-Pi-Geräte tragen eine Netzwerkkennung, die mit `b8-27-eb` beginnt. In der PowerShell erst das Netz abklopfen, dann die Antworten filtern:

```
1..254 | ForEach-Object { $null = (New-Object System.Net.NetworkInformation.Ping).SendPingAsync("192.168.$_.arp -a | findstr b8-27-eb
```

Die Liste `arp -a` zeigt nur Geräte, mit denen der PC kürzlich Kontakt hatte — ohne das vorherige Abklopfen bleibt sie unvollständig.

Fern-Desktop

`xrdp` verbindet und bricht sofort ab: Es läuft eine lokale Desktop-Sitzung, die die Grafik belegt. Abhilfe ist die Umstellung auf Textanzeige (Abschnitt 4.8).

Display

`i2cdetect -y 1` zeigt nichts: I2C eingeschaltet? Verdrahtung, besonders die Reihenfolge der Stifte am Modul. Nach dem ersten Einschalten von I2C ist ein Neustart nötig.

Streifen oder Versatz im Bild: falscher Controller, `OLED_TREIBER = "sh1106"` setzen und den Dienst neu starten.

Adresse wird gefunden, Anzeige bleibt dunkel: `journalctl -u pflanzen_waechter -n 30` ansehen. Bei einem Rechtefehler auf `/dev/i2c-1` hilft `sudo usermod -aG i2c pi` und ein Neustart.

Dienst

Dashboard nicht erreichbar: `systemctl status pflanzen_waechter` zeigt, ob der Dienst läuft; `journalctl -u pflanzen_waechter -n 50` nennt den Grund. Nach einem Absturz startet er binnen 15 Sekunden von selbst neu.

8. Was bei dieser Installation schiefging

Festgehalten, weil beides bei einer Neuinstallation wieder auftreten kann und von außen nicht zu erkennen ist.

Der Imager schrieb die Voreinstellungen nicht

Version 2.0.10 meldete „Schreiben erfolgreich“, legte aber keine `custom.toml` auf der Karte an. Der Pi startete dadurch ohne Benutzer, ohne WLAN und ohne SSH — von außen nicht von einem Verdrahtungs- oder Kennwortfehler zu unterscheiden. Der Fehler ist für mehrere Fassungen der 2.0-Reihe belegt (rpi-imager, Issues #1439, #1567 und weitere).

Verstümmelte Beschriftungen

Ebenfalls 2.0.10 unter Windows: die eingebettete Schrift wird mit verschobener Zeichentabelle geladen, jeder Buchstabe erscheint zwei Stellen versetzt. Nur Schaltflächen betroffen, Überschriften korrekt. Belegt in den Issues #1648, #1653, #1684, behoben in 2.0.11. Kein Virus und ohne Einfluss auf die geschriebenen Daten.

Die Ersteinrichtung läuft nur einmal

Ausgelöst wird sie durch einen Eintrag in `cmdline.txt` (`init=/usr/lib/raspberrypi-sys-mods/firstboot`), den das Einrichtungsprogramm anschließend selbst entfernt. Wer eine `custom.toml` erst nach dem ersten Einschalten nachträgt, wartet vergeblich — sie wird nie gelesen. Die Reihenfolge ist zwingend: brennen, Datei kopieren, dann erst zum ersten Mal einschalten.

Diese drei Punkte zusammen sind der Grund, warum die Einrichtung hier am Monitor erfolgte statt kopflos. Mit Bildschirm ist jeder dieser Fehler in Sekunden sichtbar.

9. Dateien

Im Ordner /home/pi/tomate auf dem Pi:

Datei	Inhalt
pflanzen_dashboard.py	Hauptprogramm: Bildabholung, Auswertung, Webserver, Anbindung von Display und Klemmenebene
oled_anzeige.py	Displaytreiber (SSD1306/SH1106) und Seitengestaltung
waveshare_io.py	Modbus RTU: Relais, Digital- und Analogein-/ausgänge, Gießfunktion
waveshare_pi_test.py	Prüfprogramm für das Modul A (nur bei angehaltenem Dienst)
waveshare_wandler_test.py	Prüfprogramm für den RS485-Wandler allein
pflanzen_waechter.service	Vorlage des Systemdienstes
install_pi.sh	Einrichtungsskript
ANLEITUNG_Pi_OLED.md	dieses Dokument

Im Betrieb entstehen zusätzlich:

Datei	Inhalt
pflanzen_log.csv	Messreihe: Zeit, Bildnummer, Helligkeit, Fläche, Schwerpunkt, Bewertung
pflanzen_referenz.json	die gesetzte Referenz
pflanzen_bilder/	gespeicherte Bilder mit Zeitstempel im Namen

Der eingerichtete Dienst liegt unter /etc/systemd/system/pflanzen_waechter.service.

10. Wiederaufbau von Null — wenn die Karte stirbt

Eine Speicherkarte ist ein Verschleißteil. Fällt sie aus, ist das kein Unglück, sondern eine knappe Stunde Arbeit — vorausgesetzt, die folgenden Dinge liegen außerhalb des Pi.

10.1 Was vorher gesichert sein muss

Was	Wo aufbewahren	Warum
die Programmdateien aus Abschnitt 9	auf dem PC, zusätzlich USB-Stick oder Cloud	sie liegen sonst nur auf der defekten Karte
dieses Dokument	ebenda	enthält den ganzen Weg
pflanzen_log.csv	gelegentlich auf den PC holen	die Messreihe ist nicht wiederherstellbar

Die Messreihe holt man mit einem Befehl auf den PC (in einer PowerShell auf dem PC, nicht im SSH-Fenster):

```
scp pi@<IP-des-Pi>:/home/pi/tomate/pflanzen_log.csv .
```

Verloren gehen bei einem Kartenschaden ohne Sicherung: die Messreihe, die gespeicherten Bilder und die gesetzte Referenz. Die Referenz ist der einzige Punkt, der Handlung erfordert — sie wird nach dem Wiederaufbau beim nächsten Gießen neu gesetzt. Das Programm selbst und alle Einstellungen entstehen aus den gesicherten Dateien neu.

10.2 Zeitbedarf und Material

Rund vierzig Minuten, davon zehn Minuten Warten beim Brennen. Gebraucht werden eine neue microSD-Karte (16 GB genügen, Klasse 10, für Dauerbetrieb gern eine High-Endurance-Karte), ein Kartenleser am PC und für die Ersteinrichtung Monitor, Tastatur und Maus am Pi.

Die Ersteinrichtung ohne Monitor ist möglich, aber nach den Erfahrungen aus Abschnitt 8 nicht zu empfehlen: Schlägt sie fehl, ist der Pi stumm und der Grund von außen nicht erkennbar.

10.3 Ablauf

Schritt 1 bis 3 sind ausführlich in Abschnitt 4 beschrieben, hier nur die Kurzfassung zum Abhaken.

1. Karte brennen mit Imager 1.9: Gerät Raspberry Pi 3, Betriebssystem Raspberry Pi OS (32-bit) mit Desktop, Voreinstellungen NEIN.
2. Monitor, Tastatur, Maus anschließen, Karte einlegen, Strom anlegen.
3. Begrüßungsassistenten durchgehen: Germany / German / Berlin, Benutzer pi mit Kennwort xxxxxxxxx, WLAN <WLAN-Name>, Browser Chromium, Aktualisierung überspringen. Neu starten lassen.
4. Auf dem Pi ein Terminalfenster öffnen und die Grundeinstellungen in einem Rutsch setzen — das ersetzt das Klicken in der Konfigurationsoberfläche:

```
sudo raspi-config nonint do_hostname tomate
sudo raspi-config nonint do_ssh 0
sudo raspi-config nonint do_vnc 0
sudo raspi-config nonint do_i2c 0
sudo raspi-config nonint do_boot_behaviour B2
sudo reboot
```

Die 0 bedeutet bei diesen Befehlen jeweils „einschalten“. B2 stellt den Start auf die Textanzeige um; ohne das läuft der Fern-Desktop später nicht (Abschnitt 7).

5. Adresse feststellen. Nach dem Neustart zeigt der Monitor eine Eingabeaufforderung; dort anmelden mit pi / xxxxxxxxx und eingeben:

```
hostname -I
```

Erscheint keine Adresse mit 192.168.x., hat das WLAN nicht verbunden; dann `sudo raspi-config` → System Options → Wireless LAN.

6. Vom PC aus verbinden und den Ordner anlegen:

```
ssh pi@ADRESSE
mkdir -p ~/tomate
```

7. In einer zweiten PowerShell auf dem PC, im Ordner mit den Dateien, jede einzeln übertragen (eine je Zeile, siehe Abschnitt 4.6):

```
scp pflanzen_dashboard.py pi@ADRESSE:/home/pi/tomate/
scp oled_anzeige.py pi@ADRESSE:/home/pi/tomate/
scp waveshare_io.py pi@ADRESSE:/home/pi/tomate/
scp waveshare_pi_test.py pi@ADRESSE:/home/pi/tomate/
scp waveshare_wandler_test.py pi@ADRESSE:/home/pi/tomate/
scp pflanzen_waechter.service pi@ADRESSE:/home/pi/tomate/
scp install_pi.sh pi@ADRESSE:/home/pi/tomate/
scp ANLEITUNG_Pi_OLED.md pi@ADRESSE:/home/pi/tomate/
```

8. Im SSH-Fenster einrichten:

```
cd ~/tomate
chmod +x install_pi.sh
```

```
sudo ./install_pi.sh
```

9. Fern-Desktop nachrüsten, falls gewünscht:

```
sudo apt install -y xrdp
```

10. Prüfen: `http://ADRESSE:8083` im Browser, dann Monitor und Tastatur abziehen, Strom trennen, wieder anstecken. Kommt das Dashboard nach zwei Minuten von selbst wieder, ist der Wiederaufbau abgeschlossen.

11. Nach dem nächsten Gießen im Dashboard Referenz setzen.

10.4 Wenn die Adresse eine andere ist

Der Pi bekommt seine Adresse von der Fritz!Box und kann nach einer Neuinstallation eine andere erhalten. Ermitteln lässt sie sich am Gerät mit `hostname -I` oder vom PC aus über die Netzwerkerkennung (Abschnitt 7, Unterpunkt Netzwerk). Wer Ruhe haben will, vergibt in der Fritz!Box unter Heimnetz eine feste Adresse für `tomate`.

11. Offene Punkte

- Erste Referenz nach dem Gießen setzen, sobald die Kamera wieder Strom hat
- Systemaktualisierung nachholen: `sudo apt update && sudo apt full-upgrade` (der Schritt im Begrüßungsassistenten brach ab)
- Wenn das Gerät dauerhaft draußen steht: feste Adresse in der Fritz!Box vergeben oder den Namen `tomate` verwenden, damit die Adresse nicht wandert